# Crypt: Gameplay Mechanics Specification

Punt Labs

March 2026 — v1.0

## Contents

**1 Design Principles** **2**

**2 Character Model** **2**
2.1 Attributes . . . 2
2.2 Stat Modifier . . . 2
2.3 Classes . . . 2
2.4 Point-Buy Stat Allocation . . . 2
2.5 Starting State . . . 3

**3 Experience and Leveling** **3**
3.1 XP Awards . . . 3
3.2 XP Thresholds . . . 4
3.3 Level-Up Effects . . . 4

**4 Combat** **4**
4.1 Initiative . . . 4
4.2 Hero Actions . . . 4
4.2.1 Attack . . . 4
4.2.2 Defend . . . 5
4.2.3 Flee . . . 5
4.2.4 Cast Spell . . . 5
4.3 Enemy Actions . . . 5
4.4 Damage Reduction . . . 5
4.5 Death . . . 6
4.6 Room Clearing . . . 6

**5 Spells** **6**
5.1 Casting Prerequisites . . . 6
5.2 Damage Spells . . . 6
5.3 Heal Spells . . . 6

**6 Inventory and Equipment** **6**
6.1 Item Types . . . 6
6.2 Weight Limit . . . 6
6.3 Equipment Slots . . . 7
6.4 Consumable Items . . . 7

**7 Movement and Exploration** **7**
7.1 Connections . . . 7
7.2 Room State . . . 7
7.3 Fog of War . . . 8
7.4 Combat Auto-Start . . . 8

**8 Commands** **8**

**9 Save and Load** **8**

**10 Scenario Format** **8**
10.1 Scenario Fields . . . . . 9

**11 Balance Tuning** **9**
11.1 Key Metrics . . . . . 9
11.2 Design Targets . . . . . 9

**12 Notation Reference** **10**

**13 Unimplemented Mechanics** **10**

# 1 Design Principles

All game rules are implemented deterministically in `internal/engine/`. The engine is the single source of truth for state transitions. No game rule or state mutation is left to probabilistic LLM output (DES-009).

- All randomness uses dice notation: `NdS` (roll $N$ dice with $S$ sides) with optional modifiers `NdS+M` or `NdS-M`.
- All damage floors at 1 — no attack can deal 0 damage.
- All HP caps at `MaxHP` — healing cannot exceed the maximum.
- The engine uses `math/rand` for all dice rolls. Seeding the global RNG before a session produces a fully deterministic playthrough.

# 2 Character Model

## 2.1 Attributes

Every character has six attributes, inspired by Wizardry I (DES-022).

| **Attribute** | **Abbr.** | **Mechanical Effect** |
|---|---|---|
| Strength | STR | None (reserved for damage bonus) |
| Dexterity | DEX | Initiative roll (+DEX to 1d20), flee success (1d20 $\leq$ DEX) |
| Constitution | CON | HP bonus per level-up: $\lfloor(\text{CON} - 10)/2\rfloor$ |
| Intelligence | INT | None (reserved for spell power) |
| Wisdom | WIS | None (reserved for MP regen) |
| Charisma | CHA | None (reserved for shop prices) |

Table 1: Character attributes. DEX and CON have mechanical effects; others are reserved for future systems.

## 2.2 Stat Modifier

The *stat modifier* converts a raw attribute value to a bonus or penalty. Used by the CON $\rightarrow$ HP system and reserved for future STR $\rightarrow$ damage and INT $\rightarrow$ spell power systems.

$$\text{modifier}(\text{stat}) = \left\lfloor \frac{\text{stat} - 10}{2} \right\rfloor$$

Go integer division truncates toward zero, so odd values below 10 (e.g. stat 9) produce modifier 0, not $-1$.

| **Stat** | 8 | 10 | 11 | 12 | 13 | 14 | 16 | 18 | 20 |
|---|---|---|---|---|---|---|---|---|---|
| **Modifier** | $-1$ | 0 | 0 | $+1$ | $+1$ | $+2$ | $+3$ | $+4$ | $+5$ |

Table 2: Stat modifier reference. Values below 10 are rare (base is 10, stats only increase on level-up).

## 2.3 Classes

Four classes are available. All share the same starting attributes and HP (20). Classes differ in MP access, XP progression rate, and per-level gains.

## 2.4 Point-Buy Stat Allocation

At character creation, the player distributes **8 points** across the six attributes. Each attribute starts at a base of **10**. No attribute may be set below 10. The total points spent above base must equal exactly 8.

| Class | Starting MP | HP/Level | MP/Level | Can Cast | Primary Stats |
|---|---|---|---|---|---|
| Fighter | 0 | +8 | +0 | No | STR, CON |
| Mage | 10 | +4 | +4 | Yes | INT, WIS |
| Priest | 10 | +6 | +3 | Yes | WIS, CHA |
| Thief | 0 | +6 | +0 | No | DEX, CHA |

Table 3: Class progression. Primary stats gain +1 per level-up.

$$\sum_{a \in \{\text{STR,DEX,CON,INT,WIS,CHA}\}} (a - 10) = 8, \quad a \geq 10 \; \forall \; a$$

If no stats are provided, the default allocation is used: STR 14, DEX 12, CON 12, INT 10, WIS 10, CHA 10.

| Build | STR | DEX | CON | INT | WIS | CHA |
|---|---|---|---|---|---|---|
| Default | 14 | 12 | 12 | 10 | 10 | 10 |
| Tank (fighter) | 14 | 10 | 14 | 10 | 10 | 10 |
| Agile (thief) | 12 | 14 | 12 | 10 | 10 | 10 |
| Scholarly (mage) | 10 | 12 | 10 | 14 | 12 | 10 |
| Devout (priest) | 10 | 12 | 12 | 10 | 14 | 10 |

Table 4: Example 8-point allocations. All sum to 68 ($6 \times 10 + 8$).

Stat validation is enforced by `engine.ValidateStats()` and `engine.NewCharacter()`. The daemon's `new_game` RPC accepts an optional `stats` field; if omitted, defaults are used.

In interactive mode (`cryptd serve -t`), a character creation prompt asks for name, class, and stat allocation before the game begins. The player enters 6 numbers representing points to add to each stat (e.g. `4 0 2 0 2 0` for STR 14, CON 12, DEX 12). Pressing Enter without input uses the defaults. Scripted mode (`--script`) skips the prompt and uses defaults or CLI flags.

### 2.5 Starting State

All new characters begin with:

| Field | Value |
|---|---|
| Level | 1 |
| HP / MaxHP | 20 / 20 |
| MP / MaxMP | 10 / 10 (mage, priest) or 0 / 0 (fighter, thief) |
| Gold | 0 |
| Inventory | Empty |
| Equipment | All slots empty (weapon, armor, ring, amulet) |
| Stats | Player-allocated (8-point pool, base 10 each) |

Table 5: Character creation values. Stats are player-configured via point-buy.

## 3 Experience and Leveling

### 3.1 XP Awards

XP is awarded when an enemy is killed. The award equals the enemy's `MaxHP`, not its current HP at death.

$$\text{XP}_{\text{award}} = \text{enemy.MaxHP}$$

XP is cumulative and never lost. Level-up checks occur after each combat victory (all enemies defeated). Multiple level-ups can occur in a single check if accumulated XP is sufficient.

### 3.2 XP Thresholds

Each class has a distinct XP table. Fighters level cheaply; mages require the most XP. Inspired by Wizardry I's class-based progression rates.

| Level | Fighter | Thief | Priest | Mage |
|---|---|---|---|---|
| 2 | 20 | 22 | 25 | 30 |
| 3 | 50 | 55 | 60 | 75 |
| 4 | 100 | 110 | 120 | 150 |
| 5 | 200 | 220 | 250 | 300 |
| 6 | 400 | 440 | 500 | 600 |
| 7 | 800 | 880 | 1000 | 1200 |
| 8 | 1600 | 1760 | 2000 | 2400 |
| 9 | 3200 | 3520 | 4000 | 4800 |
| 10 | 6400 | 7040 | 8000 | 9600 |

Table 6: Cumulative XP required to reach each level. Max level is 10.

### 3.3 Level-Up Effects

On level-up, effects are applied in this order:

1. **Stats:** +1 to each of the class's two primary stats (see Table 3). Applied first so the new CON value is used for the HP calculation.
2. **HP:** class base amount + CON modifier, minimum 1:

$$\text{HP gain} = \max\big(1,\ \text{hpPerLevel}[\text{class}] + \lfloor(\text{CON} - 10)/2\rfloor\big)$$

   Fighters (base 8, CON growth) gain 9–10+ HP at higher levels. Mages (base 4, no CON growth) gain 4 HP throughout.
3. **MP:** class-specific amount added to both `MP` and `MaxMP` (fighters and thieves gain 0).

## 4 Combat

Combat is turn-based and auto-starts when the hero enters a room containing enemies (unless the room has been cleared by a previous victory).

### 4.1 Initiative

At combat start, initiative determines turn order:

$$\text{Hero initiative} = \text{DEX} + \texttt{1d20} \tag{1}$$

$$\text{Enemy initiative} = 10 + \texttt{1d20} \quad \text{(per enemy)} \tag{2}$$

Turn order is sorted descending by initiative roll. Ties are broken in favor of the hero. Once set, turn order is fixed for the duration of combat. Dead enemies are skipped during turn processing.

### 4.2 Hero Actions

On the hero's turn, one of four actions may be taken:

#### 4.2.1 Attack

The hero attacks a target enemy (defaults to the first alive enemy if no target is specified).

$$\text{damage} = \begin{cases} \text{Roll(weapon.Damage)} & \text{if weapon equipped} \\ \texttt{1d2} & \text{unarmed} \end{cases}$$

Damage is applied directly to the target's HP. Minimum damage is 1. If the target's HP drops to 0 or below, the target is killed and the hero is awarded XP equal to the target's `MaxHP`. If all enemies are dead, combat ends and the room is marked `Cleared`.

#### 4.2.2 Defend

The hero assumes a defensive stance. Incoming damage from all enemies is halved (integer division, floor 1) until the hero's next turn. The `HeroDefending` flag is cleared when the hero's turn comes around again.

#### 4.2.3 Flee

The hero attempts to escape combat.

$$\text{Roll } \texttt{1d20}: \quad \text{success if roll} \leq \text{DEX}$$

With the default DEX of 12, the base flee rate is $12/20 = 60\%$. Thieves gain +1 DEX per level, improving their flee rate.

- **Success:** Combat ends. The room is *not* marked cleared — enemies will attack again if the hero re-enters.
- **Failure:** The hero loses their turn. Enemies still act.

#### 4.2.4 Cast Spell

See Section 5.

### 4.3 Enemy Actions

After the hero acts, all enemies take their turns in initiative order before the hero acts again. Each enemy follows its AI pattern:

| AI | Behavior |
|---|---|
| `aggressive` | Always attacks. |
| `cautious` | Attacks normally. Flees (removed from combat) when HP $\leq 30\%$ of `MaxHP`. Flee awards no XP. |
| `scripted` | Alias for aggressive. |

Table 7: Enemy AI patterns.

Enemy attack damage:

$$\text{raw damage} = \text{Roll(enemy.Attack)}$$

Damage is then reduced by defenses (see Section 4.4).

### 4.4 Damage Reduction

When an enemy attacks the hero, raw damage is reduced by two mechanisms applied in order:

1. **Defend stance:** If the hero is defending, damage $= \lfloor \text{damage}/2 \rfloor$.
2. **Armor defense:** If the hero has armor equipped with a `defense` value $d > 0$, damage $=$ damage$-$ $d$.

After both reductions, damage is floored at 1:

$$\text{final damage} = \max\big(1,\ \lfloor \text{raw}/k \rfloor - d\big) \quad \text{where } k = \begin{cases} 2 & \text{defending} \\ 1 & \text{otherwise} \end{cases}$$

### 4.5 Death

When the hero's HP drops to 0 or below during enemy turns, the hero dies. The game loop terminates immediately after narrating the death event. The scenario's `death` field (`respawn` or `permadeath`) determines whether the game can be reloaded, but the engine does not currently implement respawn mechanics.

### 4.6 Room Clearing

When all enemies in a room are killed (HP $\leq 0$), the room's `Cleared` flag is set to `true`. A cleared room will not trigger combat on re-entry. Enemies that flee (cautious AI) are removed from combat; if all remaining enemies are dead or fled, the room is cleared and the hero is awarded XP for kills (not for fled enemies).

## 5 Spells

Spells are defined in scenario YAML with five fields: `name`, `mp` (cost), `effect` (`damage` or `heal`), `power` (dice notation), and `classes` (list of allowed classes).

### 5.1 Casting Prerequisites

A spell cast fails (and costs no MP) if any of the following are true:

1. The hero is dead (HP $\leq 0$).
2. The spell ID does not exist in the scenario.
3. The hero's class is not in the spell's `classes` list.
4. The hero's MP is less than the spell's `mp` cost.
5. (Damage spells only) Combat is not active.
6. (Damage spells only) It is not the hero's turn.
7. (Damage spells only) The target is invalid (dead or nonexistent).

MP is deducted *after* the spell effect resolves, never on failure.

### 5.2 Damage Spells

Damage spells require active combat and a valid target. The spell's `power` is rolled as dice notation.

$$\text{spell damage} = \max(1,\ \text{Roll}(\text{spell.Power}))$$

The damage is applied directly to the target's HP. If the target dies, XP is awarded (equal to target's `MaxHP`) and combat may end. Casting a damage spell consumes the hero's turn.

### 5.3 Heal Spells

Heal spells work both in and out of combat. When cast in combat, they consume the hero's turn. The spell's `power` is rolled as dice notation and added to the hero's HP, capped at `MaxHP`.

$$\text{hero.HP} = \min(\text{MaxHP},\ \text{hero.HP} + \max(1,\ \text{Roll}(\text{spell.Power})))$$

## 6 Inventory and Equipment

### 6.1 Item Types

Items have a `type` field that determines how they can be used:

### 6.2 Weight Limit

Every character has a carry weight limit of **50.0** units. Total weight includes both inventory items and equipped items. Picking up an item that would exceed the limit returns a `TooHeavyError`.

| Type | Slot | Behavior |
|---|---|---|
| `weapon` | Weapon | Equipped to weapon slot. `damage` field used for attack rolls. |
| `armor` | Armor | Equipped to armor slot. `defense` field reduces incoming damage. |
| `ring` | Ring | Equipped to ring slot. No mechanical effect (reserved). |
| `amulet` | Amulet | Equipped to amulet slot. No mechanical effect (reserved). |
| `consumable` | — | Used via `use` command. Consumed on use. `effect` and `power` determine the effect. |
| `key` | — | Carried in inventory. No use action (reserved for locked doors). |
| `misc` | — | Carried in inventory. No mechanical effect. |

Table 8: Item types and their equipment behavior.

### 6.3 Equipment Slots

Four equipment slots exist: weapon, armor, ring, amulet. Each slot holds at most one item. Equipping an item removes it from inventory and places it in the slot. Unequipping moves it back to inventory. Attempting to equip into an occupied slot returns a `SlotOccupiedError`.

### 6.4 Consumable Items

Consumable items are used via the `use` command. On use, the item is removed from inventory and its effect is applied:

- **Heal:** Roll `power` dice, add to hero HP (capped at `MaxHP`). Can be used in or out of combat.

Using a non-consumable item returns a `NotConsumableError`. Using consumables does not consume a combat turn (it is processed outside the combat turn system).

## 7 Movement and Exploration

### 7.1 Connections

Rooms are connected by directional exits. Six directions are supported: north, south, east, west, up, down. Each connection has a type:

| Type | Behavior |
|---|---|
| `open` | Freely traversable. |
| `locked` | Returns `LockedError`. Cannot be traversed (key mechanics not yet implemented). |
| `hidden` | Invisible to the player. Not listed in exits. Returns `NoExitError` if attempted. |
| `stairway` | Freely traversable (functionally identical to `open`). |

Table 9: Connection types.

### 7.2 Room State

Each room has mutable state tracking:

- `Cleared`: whether all enemies have been defeated. A cleared room does not trigger combat on re-entry.
- `Items`: list of item IDs currently in the room. Items can be picked up (removed from room, added to inventory) or dropped (added to room from inventory). Room items are seeded from the scenario on `NewGame`.

### 7.3 Fog of War

The `VisitedRooms` list tracks which rooms the hero has entered. Rooms not in this list are unvisited. The list grows monotonically — rooms are never removed.

### 7.4 Combat Auto-Start

When the hero moves into a room that has enemies and is not cleared, `StartCombat` is called automatically after the move resolves. The hero sees the room description, then immediately enters combat. If enemies win initiative, they act before the hero.

## 8 Commands

The `RulesInterpreter` maps text input to engine actions. All commands are case-insensitive. Articles (*the*, *a*, *an*) are stripped. Multi-word targets are joined with underscores. Typos within edit distance 1 of known verbs are auto-corrected.

| **Input** | **Aliases** | **Action** |
|---|---|---|
| `north`, `south`, etc. | `n`, `s`, `e`, `w`, `u`, `d` | Move in direction |
| `go <dir>` | — | Move in direction |
| `ascend` | — | Move up |
| `descend` | — | Move down |
| `look` | `l` | Describe current room |
| `examine <item>` | `x` | Inspect an item |
| `look at <item>` | — | Alias for examine |
| `take <item>` | `get`, `grab` | Pick up from room |
| `pick up <item>` | — | Pick up from room |
| `drop <item>` | — | Drop to room |
| `equip <item>` | `wear`, `wield` | Equip from inventory |
| `unequip <slot>` | `remove` | Unequip to inventory |
| `use <item>` | `drink`, `eat` | Use consumable item |
| `attack [target]` | `a`, `hit`, `strike`, `kill` | Attack enemy (combat only) |
| `defend` | `block`, `guard` | Defensive stance (combat only) |
| `flee` | `run`, `escape` | Attempt to flee (combat only) |
| `cast <spell>` | — | Cast a spell |
| `cast <spell> at <target>` | — | Cast damage spell at target |
| `inventory` | `i` | List inventory and equipment |
| `save [slot]` | — | Save game (default: quicksave) |
| `load [slot]` | — | Load game (default: quicksave) |
| `help` | `?` | List available commands |
| `quit` | `exit`, `q` | End the game |

Actions blocked during combat: `move`, `take`, `drop`, `equip`, `unequip`, `examine`. Attempting them returns an `in_combat` event.

## 9 Save and Load

The game supports named save slots (default: `quicksave`). The full `GameState` is serialized to JSON with a `schema_version` field. Unknown fields in save files are silently ignored for forward compatibility. Loading replaces the entire in-memory game state.

Save files are stored in `.dungeon/saves/<slot>.json`.

## 10 Scenario Format

Scenarios are defined in YAML. Two formats are supported:

1. **Single-file:** one `.yaml` file with all rooms, items, enemies, and spells.

2. **Directory:** a directory containing `scenario.yaml` (manifest with title, starting room, catalogs, and region list) plus `regions/*.yaml` sub-files. Room IDs are globally unique across regions.

Scenario resolution (`scenariodir.Load`): tries directory format first (`id/scenario.yaml`), falls back to single-file (`id.yaml`).

### 10.1 Scenario Fields

| Field | Type | Description |
|---|---|---|
| `title` | string | Display name |
| `starting_room` | string | Room ID where the adventure begins |
| `death` | string | `respawn` or `permadeath` |
| `rooms` | map | Room definitions keyed by ID |
| `items` | map | Item templates keyed by ID |
| `enemies` | map | Enemy templates keyed by ID |
| `spells` | map | Spell templates keyed by ID |

Table 11: Top-level scenario fields.

## 11 Balance Tuning

The `eval-balance` tool runs Monte Carlo simulations using a monkey-testing harness (`internal/monkeytest`). The monkey player makes weighted-random decisions based on game state:

| State | Attack | Defend | Flee | Heal |
|---|---|---|---|---|
| Combat, HP $>$ 30% | 70% | 15% | 10% | 5%* |
| Combat, HP $\leq$ 30% | 20% | 20% | 40% | 20%* |

Table 12: Combat action weights. *Heal only available to casters with sufficient MP; otherwise weight redistributed.

Out of combat, the monkey auto-equips weapons and armor, uses health potions at $\leq$ 50% HP, picks up items (60%), moves randomly (30%), or looks around (10%).

### 11.1 Key Metrics

Per-session metrics collected:

2

- Total moves
- Rooms visited (unique)
- Enemies killed
- XP gained
- Final level
- Levels gained
- Items picked up / equipped
- Combat rounds fought
- Damage dealt / taken
- Survival (alive at end)
- Flee attempts / successes
- Spells cast / heals cast
- Potions used

Aggregate report includes: survival rate, mean/median/p95 for all metrics, level-up rate, per-class breakdowns. Output is JSON to stdout.

### 11.2 Design Targets

Target ranges for a well-balanced scenario:

| Metric | Target Range |
|---|---|
| Survival rate (all classes) | 50–75% |
| Class survival spread (max − min) | $< 20$ percentage points |
| Level-up rate | $> 50\%$ |
| Mean enemies killed | $\geq 3$ |
| Flee success rate | 55–70% (consistent with DEX 12) |

Table 13: Balance targets for monkey testing. A scenario that falls outside these ranges needs tuning.

# 12 Notation Reference

| Notation | Meaning |
|---|---|
| `NdS` | Roll $N$ dice, each with $S$ sides. Sum the results. |
| `NdS+M` | Roll $N$d$S$ and add modifier $M$. |
| `NdS-M` | Roll $N$d$S$ and subtract modifier $M$. |
| `1d20` | Standard 20-sided die roll (range 1–20). |
| `1d2` | Unarmed damage (range 1–2). |
| `1d6` | Standard weapon damage (short sword). |
| `2d6` | Spell damage (fireball). |
| `1d6+2` | Spell heal (heal spell). |

Table 14: Dice notation used throughout the engine.

# 13 Unimplemented Mechanics

The following mechanics are defined in the data model but have no engine implementation:

1. STR modifier to melee damage (stored, leveled, not used).
2. INT modifier to spell power (stored, leveled, not used).
3. WIS modifier to MP regen or spell accuracy.
4. CHA modifier to shop prices or NPC interactions.
5. Per-class starting stat *defaults* (currently all classes use the same default if no stats are provided; class-specific defaults could guide new players).
6. Key items unlocking locked doors.
7. Respawn mechanics (the `death` field is parsed but not acted on).
8. Ring and amulet mechanical effects.
9. Accuracy / miss chance (all attacks hit).
10. Critical hits.
11. Status conditions (`Conditions` field exists on `Character` and `EnemyInstance` but is never read).